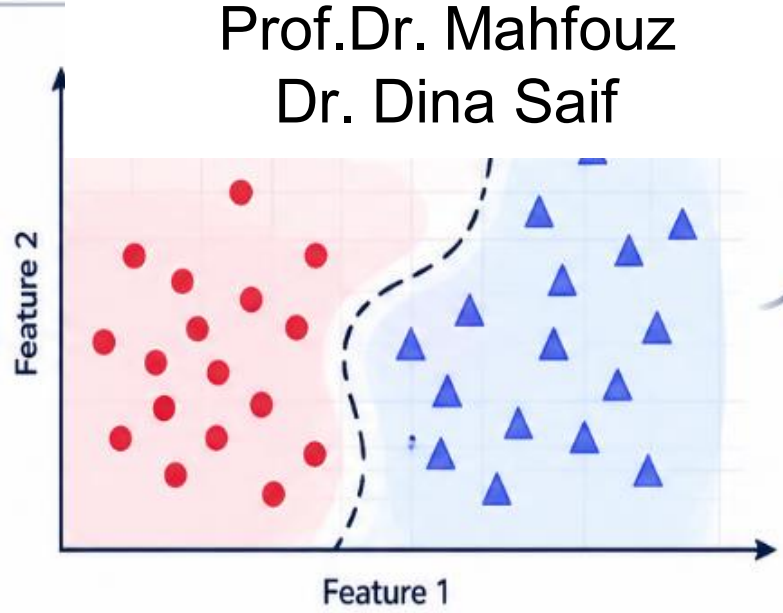
Machine Learning and Pattern Recognition

L03: Classification

Prof.Dr. Mahfouz
Dr. Dina Saif

Training Data

Feature 1	Feature 2	Label
2.1	3.5	●
1.8	2.9	●
3.2	1.7	●
6.1	2.3	▲
7.4	3.1	▲
...	...	...

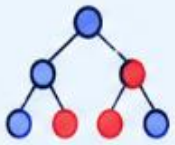


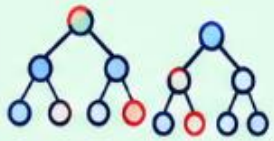
New Prediction

Feature 1 = 5.6
Feature 2 = 2.8

Predicted Class: ▲


Logistic Regression


Decision Trees


Random Forest


Support Vector Machines


Neural Networks

and more...

Outline

MNIST Dataset
Training a Binary Classifier
Performance Measures
- Measuring Accuracy Using Cross-Validation
- Confusion Matrix
- Precision and Recall
- Precision/Recall Trade-off
- The ROC Curve

Multiclass Classification
Error Analysis
Multilabel Classification
Multioutput Classification

DATASET BENCHMARK

MNIST turns handwritten digits into a learning problem

1. **70,000 labeled images** of human-written digits across ten classes (0 through 9).

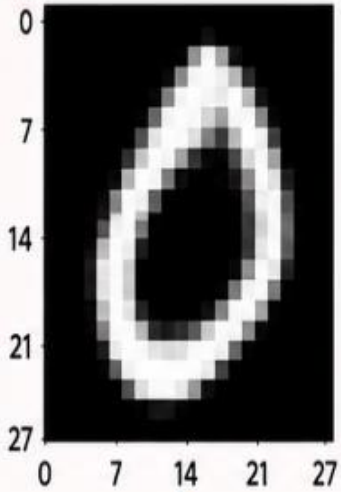
2. **28 × 28 pixels** per sample, yielding an input vector of 784 intensity features.



784 dimensions · 70k samples

Examples from the MNIST Dataset

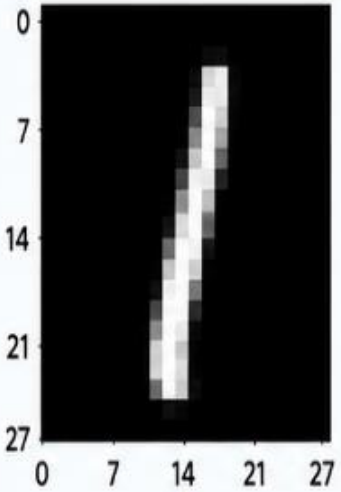
Label: 0



Pixel values (28 × 28)

0	0	0	0	0	...	0
0	0	0	0	0	...	0
0	0	5	173	255	...	0
0	0	26	255	255	...	0
0	0	186	255	255	...	0
...	...	...	...	...	...	...
0	0	0	0	0	...	0

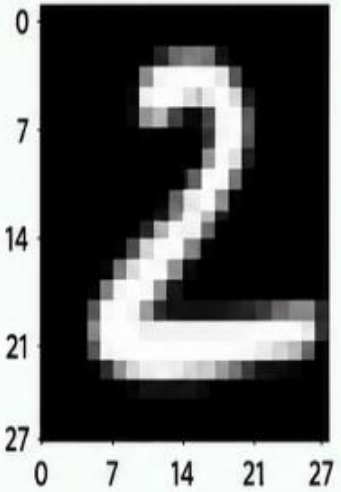
Label: 1



Pixel values (28 × 28)

0	0	0	0	0	...	0
0	0	0	0	0	...	0
0	0	0	40	254	...	0
0	0	0	182	253	...	0
0	0	0	254	254	...	0
...	...	...	...	...	...	...
0	0	0	0	0	...	0

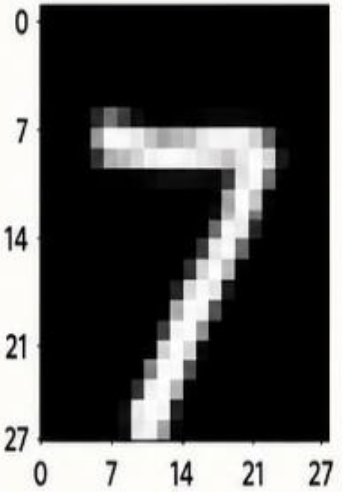
Label: 2



Pixel values (28 × 28)

0	0	0	0	0	...	0
0	0	0	0	0	...	0
0	0	18	219	255	...	0
0	0	118	255	255	...	0
0	0	237	255	255	...	0
...	...	...	...	...	...	...
0	0	0	0	0	...	0

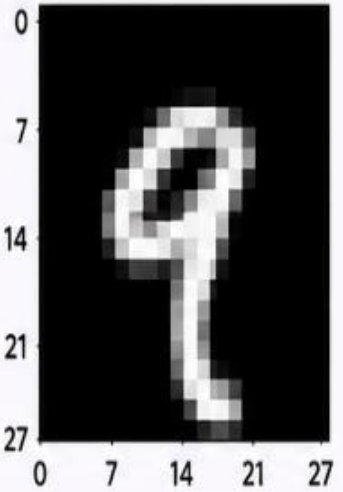
Label: 7



Pixel values (28 × 28)

0	0	0	0	0	...	0
0	0	0	0	0	...	0
0	14	186	255	255	...	0
0	0	53	255	255	...	0
0	0	0	69	255	...	0
...	...	...	...	...	...	...
0	0	0	0	0	...	0

Label: 9



Pixel values (28 × 28)

0	0	0	0	0	...	0
0	0	0	0	0	...	0
0	0	76	255	255	...	0
0	2	187	255	255	...	0
0	0	35	241	255	...	0
...	...	...	...	...	...	...
0	0	0	0	0	...	0

Each MNIST image is a 28 × 28 grayscale image, represented as a matrix of pixel values (0 = black, 255 = white).

Importing the MNIST Dataset

```
>>> from sklearn.datasets import fetch_openml
>>> mnist = fetch_openml('mnist_784', version=1)
>>> mnist.keys()
dict_keys(['data', 'target', 'feature_names', 'DESCR', 'details',
           'categories', 'url'])
```

Unpacking the Data Structure

DESCR

The Metadata

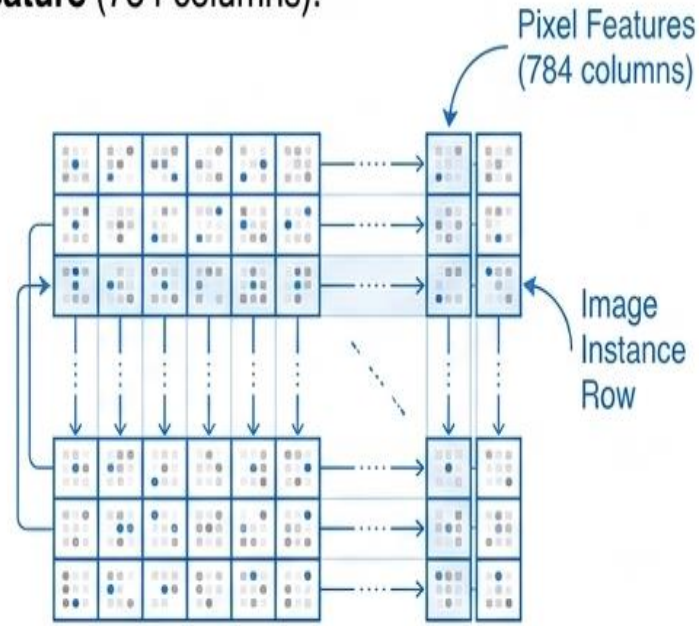
Contains the full description and background context of the dataset.



data

The Input Matrix

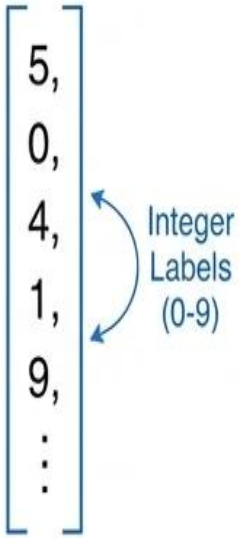
A massive array containing one row per image instance, and one column for every pixel **feature** (784 columns).



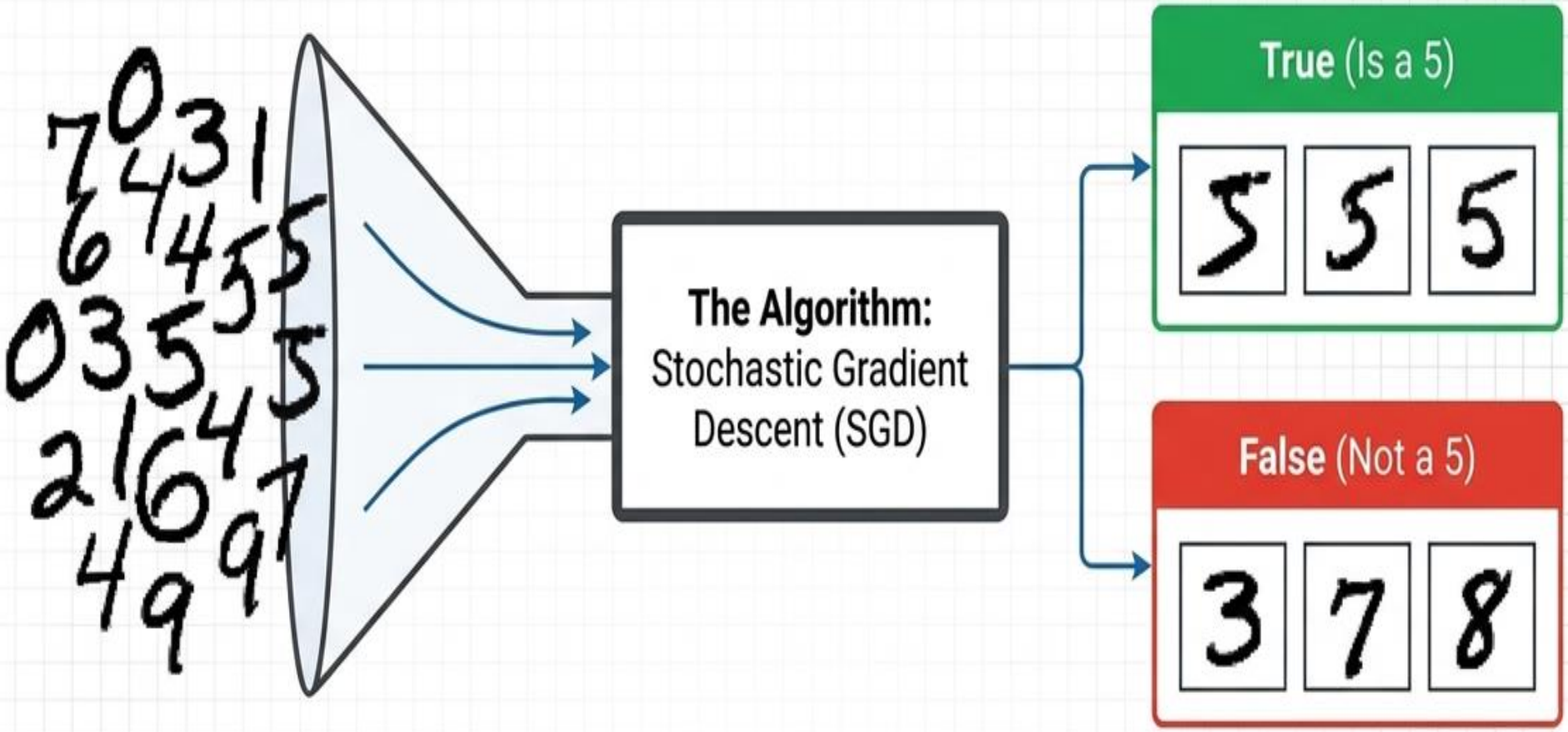
target

The Ground Truth

A 1D array containing the actual integer labels (0-9) assigned to each image.



The Binary Baseline: Isolating the '5'



Why SGD? It handles massive datasets efficiently by processing training instances independently, one at a time. This makes it exceptionally well-suited for online learning.

What is the Gradient and Why Does It Change?

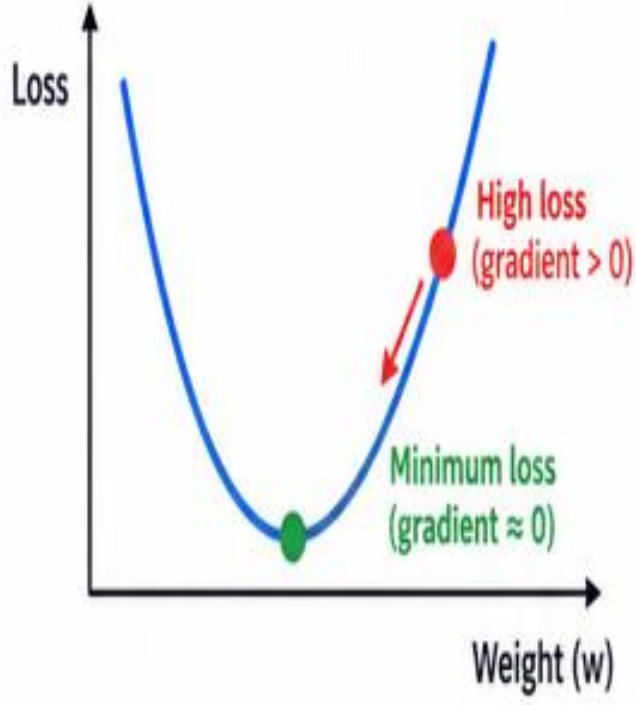
1 What is the Gradient?

The gradient tells us:

- How the loss (error) changes if we change a weight a little bit.
- The direction to move the weight.
- The size of the change.

We move in the opposite direction of the gradient to reduce the loss.

2 Visual Intuition



3 How the Gradient Changes

The gradient depends on:

1. **The input data (image)**
   Different images (e.g., digit 7 or 3) give different gradients.
2. **The current weights**
   As the weights are updated, the gradient changes.
3. **The loss function**
   The formula used to measure the error.

Training a Binary Classifier

- Let's simplify the problem for now and only try to identify one digit ⟹ number 5.

```
y_train_5 = (y_train == 5)  # True for all 5s, False for all other digits
y_test_5 = (y_test == 5)
```

- Use Stochastic Gradient Descent (SGD) classifier (**Scikit-Learn's SGDClassifier class**):
  - It has the advantage of being capable of handling very large datasets efficiently.
  - SGD deals with training instances independently, one at a time (which makes SGD well suited for online learning).

SGD Code

```
from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
>>> sgd_clf.predict([some_digit])
array([ True])
```

- The SGDClassifier relies on randomness during training (hence the name "stochastic").
- The classifier guesses that this image represents a 5 (True).

The Testing Pipeline: K-Fold Cross Validation



Mechanism

The algorithm splits the training data into "K" distinct folds. It trains the model on K-1 folds, and tests its predictions on the remaining blind fold. This cycle repeats until every fold has been tested.

Tech Note

StratifiedKFold ensures each fold contains a representative ratio of every class, preventing imbalanced training batches.

Measuring Accuracy Using Cross-Validation

- The StratifiedKFold class performs stratified sampling to produce folds that **contain a representative ratio of each class.**

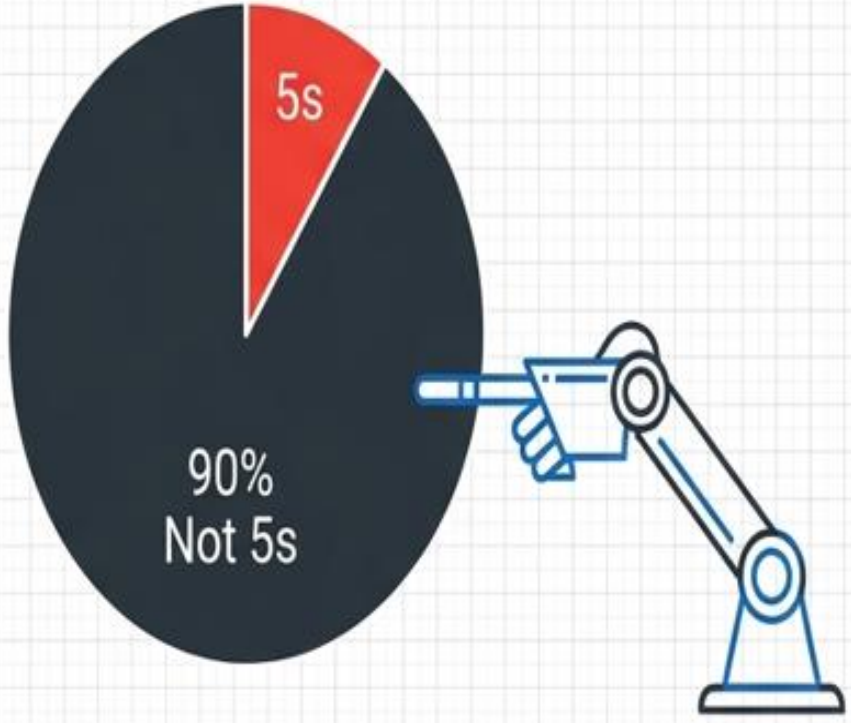
- K-fold cross-validation means splitting the **training set into K folds**, then making predictions and evaluating them on each fold using a model trained on the remaining folds.
- CV=3-Fold Cross-Validation

```
>>> from sklearn.model_selection import cross_val_score
>>> cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.96355, 0.93795, 0.95615])
```

Accuracy for each fold

The Illusion of Accuracy





The Trap

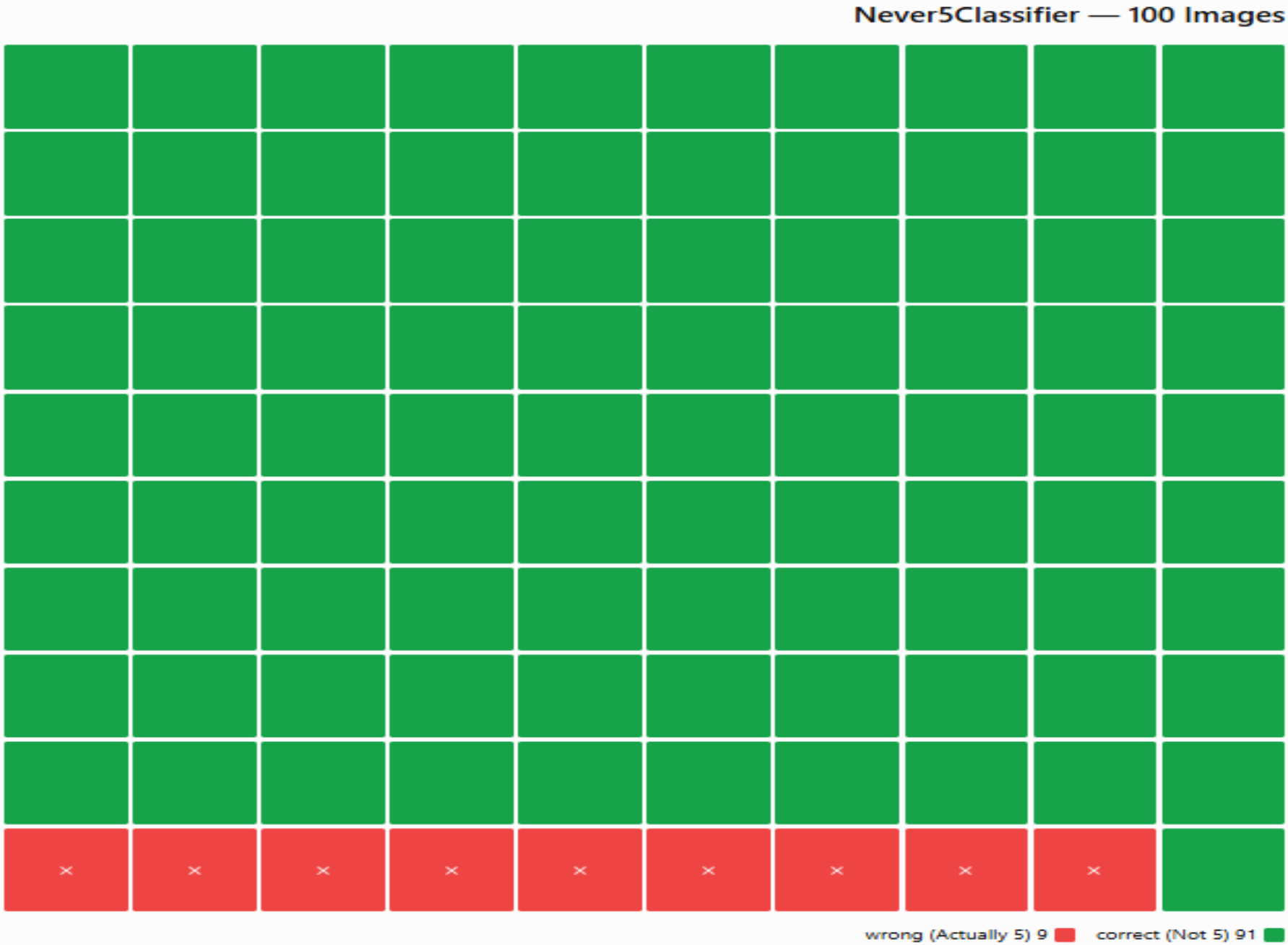
In a skewed dataset, simple accuracy lies.

The Scenario

If only 10% of our images are 5s, a completely 'dumb' classifier that simply guesses 'Not a 5' every single time will achieve exactly 90% accuracy.

The Takeaway

Accuracy measures raw hit-rate, making it the wrong tool for evaluating unbalanced data. We need a diagnostic engine.



$$Accuracy = \frac{91}{100} = \boxed{91\%}$$

Measuring Accuracy Using Cross-Validation (cont'd)

```
from sklearn.base import BaseEstimator

class Never5Classifier(BaseEstimator):
    def fit(self, X, y=None):
        return self
    def predict(self, X):
        return np.zeros((len(X), 1), dtype=bool)
>>> never_5_clf = Never5Classifier()
>>> cross_val_score(never_5_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.91125, 0.90855, 0.90915])
```

This demonstrates **why accuracy is generally not the preferred performance measure for classifiers**, especially when you are dealing with skewed datasets.

Explanation of the Example

1 Import the base class

```
from sklearn.base import BaseEstimator
```

BaseEstimator is the base class in scikit-learn that provides basic functionality for custom models (e.g., parameter handling).

2 Define the custom classifier

```
class Never5Classifier(BaseEstimator):
    def fit(self, X, y=None):
        return self
    def predict(self, X):
        return np.zeros((len(X), 1), dtype=bool)
```

- Never5Classifier is a very simple (dummy) classifier.
- fit just returns self (nothing is learned).
- predict returns all zeros (False) for every input sample.
- So, this model always predicts the class 0 for every image, no matter what the input is.

3 Create the model and evaluate with 5-fold cross-validation

```
never_5_clf = Never5Classifier()
cross_val_score(never_5_clf, X_train, y_train_5, cv=3, scoring="accuracy")
```

- Create an instance of the dummy classifier.
- Use cross_val_score to evaluate it using 3-fold (or 5-fold) cross-validation.
- In each fold, the model is trained on part of the training data and tested on the remaining part, and the accuracy is computed.

4 What does the model predict and why does it get ~0.91 accuracy?

The model always predicts 0:



→ **Predicted: 0** (for every image)

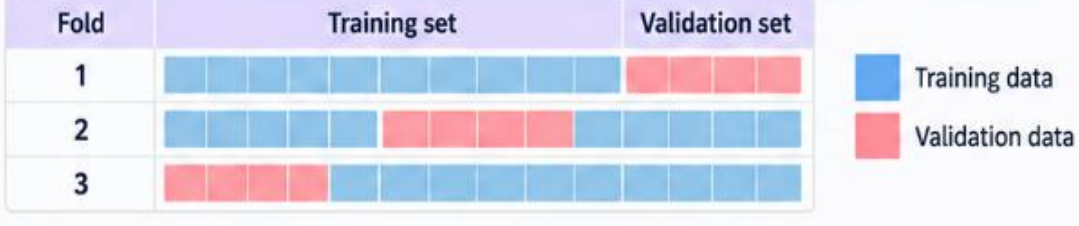
In MNIST, the digit 5 appears in about 9% of the training data.



So, if we always predict 0:

- We are correct for most images that are not 5.
- We are wrong only for the images that are 5.
- Therefore, the accuracy is about 91%.

5 Visualizing 5-fold cross-validation (example for cv=3)

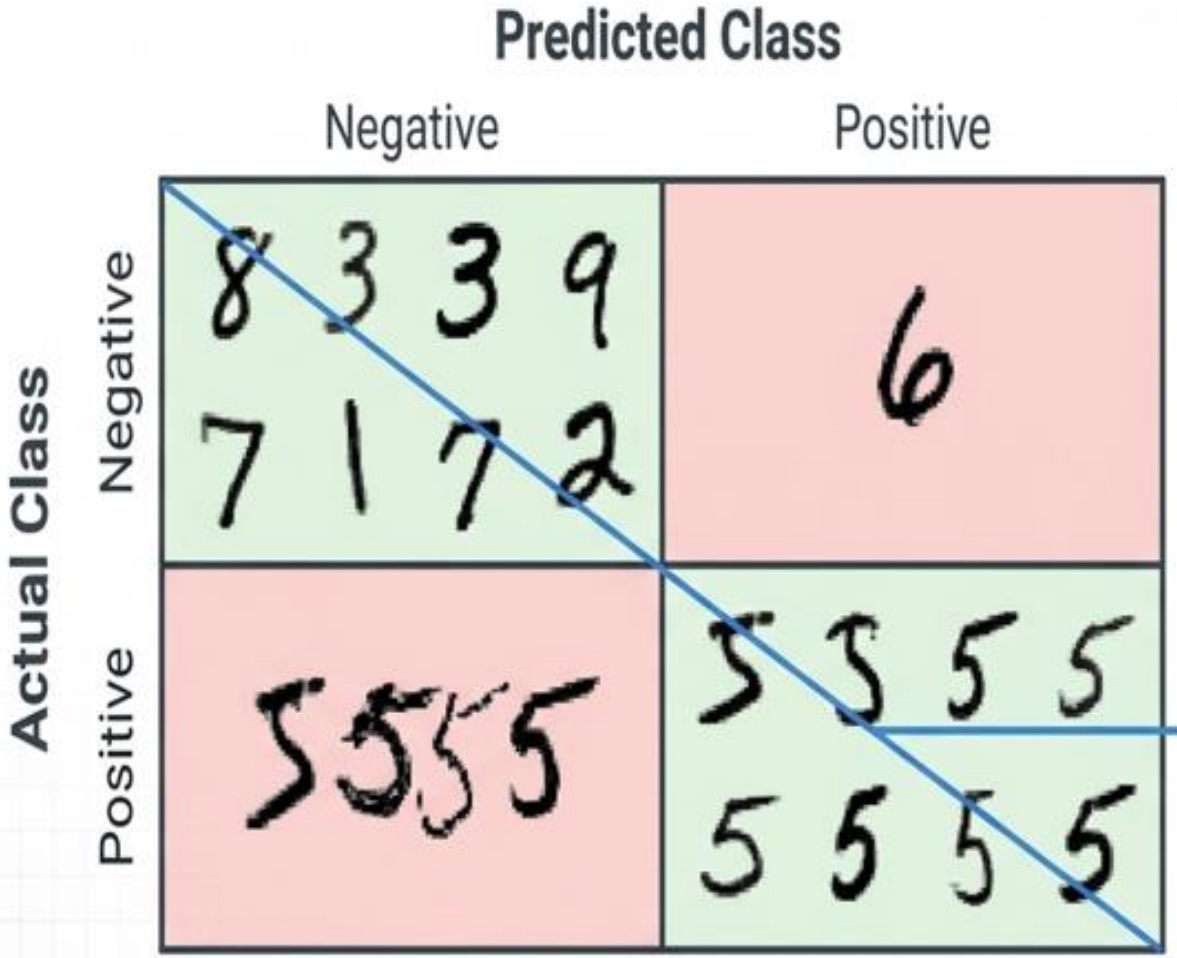


6 Output of the code

```
array([0.91125, 0.90855, 0.90915])
```

These are the accuracy scores for each fold. They are around 0.91 (91%), which means the model correctly predicts the majority class (0) in most cases.

The Diagnostic Engine: The Confusion Matrix



Layout Logic:
Rows = Actual Class.
Columns = Predicted Class.

Main Diagonal:
A perfect classifier operates entirely on this diagonal—only registering True Positives and True Negatives.

The Goal: Stop asking "How many did I get right?" Start asking "How exactly did I fail?"

True Positive (TP)



Actual: 5. Predicted: 5.
A direct hit.

False Positive (FP)



Actual: messy 3. Predicted: 5.
Type I Error: A false alarm.

False Negative (FN)



Actual: weird 5. Predicted: Not 5.
Type II Error: A missed detection.

True Negative (TN)



Actual: 3. Predicted: Not 5.
Correctly ignored.

Confusion Matrix

- A much better way to evaluate the performance of a classifier is to look at the confusion matrix.
- The general idea is to count the number of times instances of class A are classified as class B.
- Just like the cross_val_score() function, cross_val_predict() **performs K-fold cross-validation**, but instead of returning the evaluation scores, it returns the predictions made on each test fold.

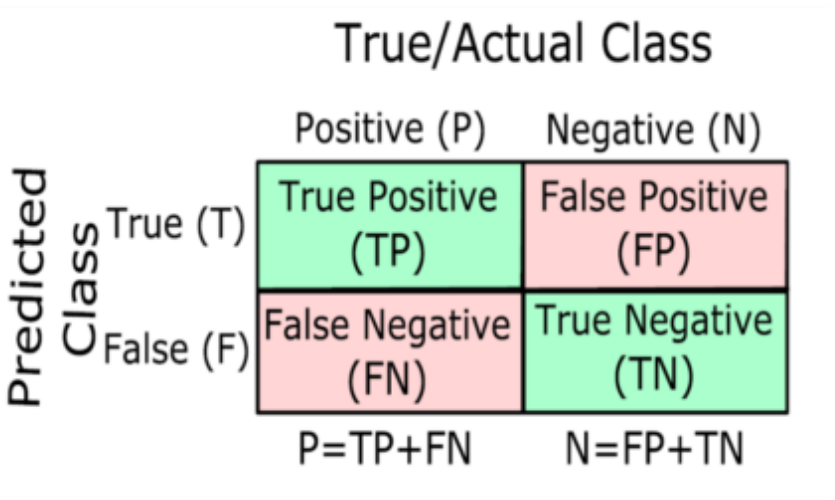
Confusion Matrix Code

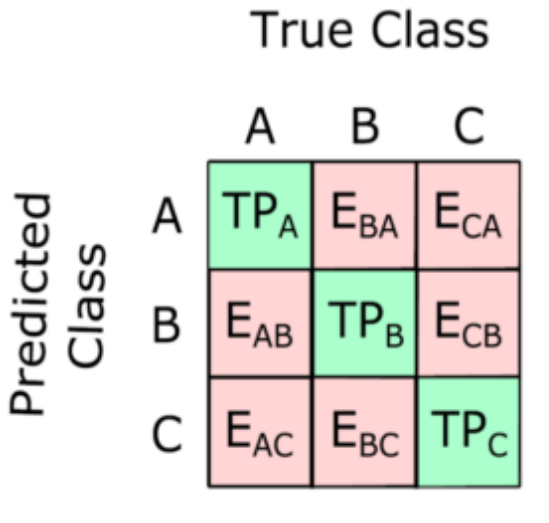
```
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
>>> from sklearn.metrics import confusion_matrix
>>> confusion_matrix(y_train_5, y_train_pred)
array([[53057,  1522],
       [ 1325,  4096]])
>>> y_train_perfect_predictions = y_train_5  # pretend we reached perfection
>>> confusion_matrix(y_train_5, y_train_perfect_predictions)
array([[54579,     0],
       [    0,  5421]])
```

TP FP FN TN

Confusion Matrix (cont'd)

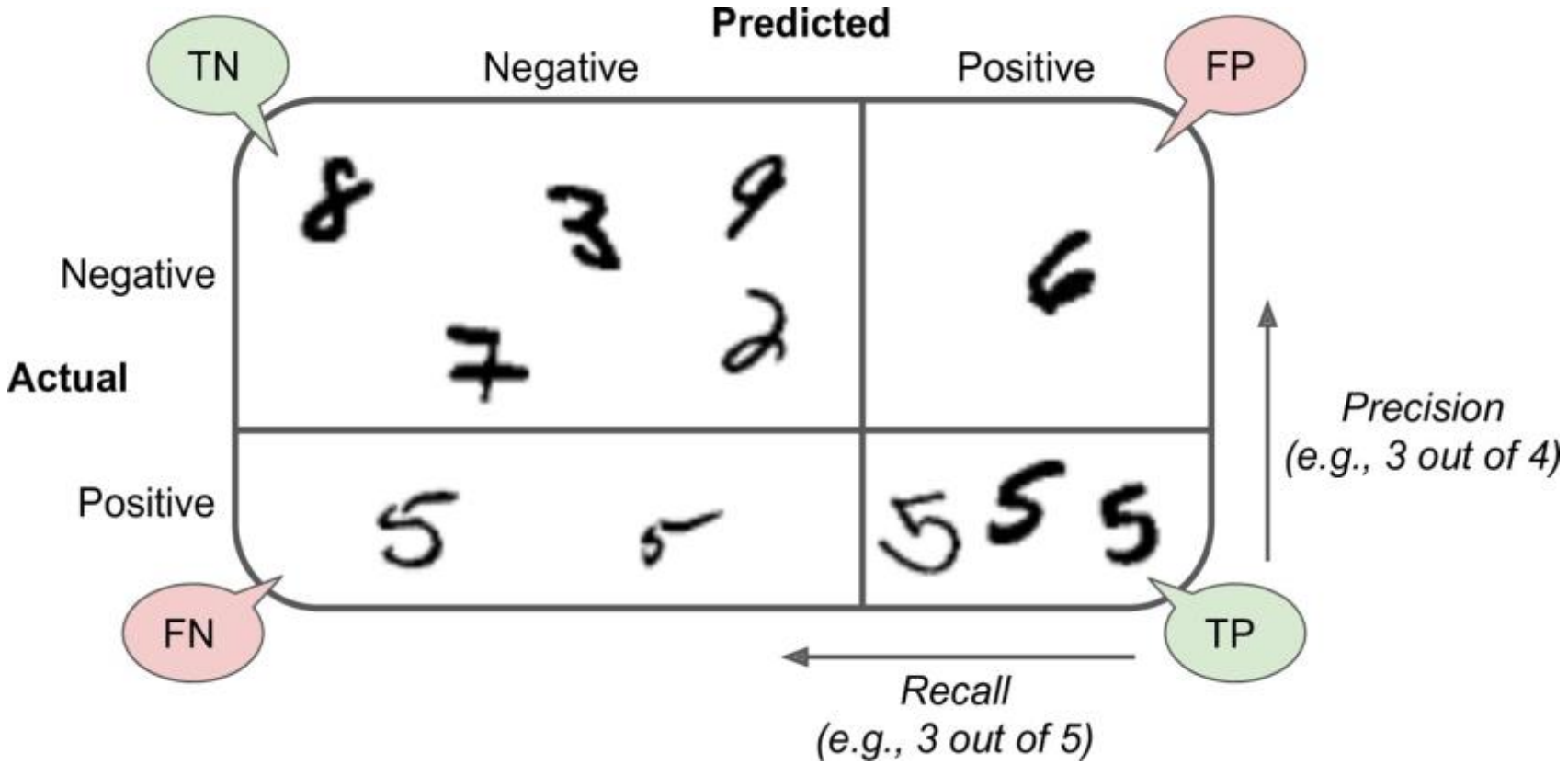




- **TP= True Positive**
- **FP= False Positive= Type I Error**
- **FN= False Negative= Type II Error**
- **TN=True Negative**

- $FP_A=E_{BA}+E_{CA}$
- $FN_A=E_{AB}+E_{AC}$

An illustrated confusion matrix



Notes on Confusion Matrix

- Each row in a confusion matrix represents an actual class, while each column represents a predicted class.
- A perfect classifier would have only **true positives** and **true negatives**, so its confusion matrix would have **nonzero values only on its main diagonal** (top left to bottom right)

Classification Metrics

- Accuracy $(ACC) = \frac{TP+TN}{TP+TN+FP+FN}$: It is defined as a ratio between the correctly classified samples to the total number of samples.
- Error Rate $(ERR) = 1 - ACC$: It represents the number of misclassified samples from both positive and negative classes.
- Sensitivity = True Positive Rate (TPR)= Hit Rate = Recall=$\frac{TP}{TP+FN}$: It represents the positive correctly classified samples to the total number of positive samples.

Classification Metrics (cont'd)

- Specificity = True Negative Rate (TNR) = $\frac{TN}{FP+TN}$: It represents the ratio of the correctly classified negative samples to the total number of negative samples.
- Positive Prediction Value (PPV) = Precision = $\frac{TP}{FP+TP}$: It represents the proportion of positive samples that were correctly classified to the total number of positive predicted samples.
- Negative Predictive Value (NPV) = $\frac{TN}{FN+TN}$: It is the inverse precision.

Confusion Matrix Example

Binary classification: Is the image **a 5** or **not**?

Actual Label		Predicted Label: Predicted Not 5 (0)	Predicted Label: Predicted 5 (1)
	Actual Not 5 (0)	**TN** **50** (correct)	**FP** **10** (error)
	Actual 5 (1)	**FN** **5** (error)	**TP** **35** (correct)

What do these values mean?

True Positive (TP)



The image is **5** and the model predicts **5**.

35 images

False Positive (FP)



The image is not 5 but the model predicts 5.

10 images

True Negative (TN)



The image is not 5 and the model predicts not 5.

50 images

False Negative (FN)



The image is 5 but the model predicts not 5.

5 images

1 Youden's Index (YI)

Measures the model's ability to distinguish between the two classes (overall discriminative power).

$$YI = TPR + TNR - 1$$

$$TPR = \frac{TP}{TP + FN} \qquad TNR = \frac{TN}{TN + FP}$$

$$TPR = \frac{35}{35 + 5} = 0.875 \qquad TNR = \frac{50}{50 + 10} = 0.833$$

$$YI = 0.875 + 0.833 - 1 = \mathbf{0.708}$$

Benefit:
High value (close to 1) means strong ability to separate 5 from not 5.

2 Matthews Correlation Coefficient (MCC)

Measures the correlation between actual and predicted classifications. Works well even with imbalanced data.

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

$$MCC = \frac{35 \times 50 - 10 \times 5}{\sqrt{45 \times 40 \times 60 \times 55}}$$

$$= \frac{1700}{\sqrt{5{,}940{,}000}} = \mathbf{0.702}$$

Benefit:
Value ranges from −1 to 1. 1 = perfect, 0 = no correlation, −1 = completely wrong.

3 F-measure (F1-score)

Harmonic mean of precision and recall. Balances finding 5s and avoiding false alarms.

$$F1 = \frac{2TP}{2TP + FP + FN}$$

$$F1 = \frac{2 \times 35}{2 \times 35 + 10 + 5}$$

$$= \frac{70}{85} = \mathbf{0.824}$$

Benefit:
High value means both precision and recall are high (good balance).

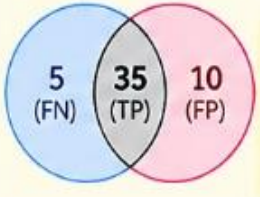
4 Jaccard Index (Jaccard / IoU)

Measures the overlap between actual 5s and predicted 5s. Focuses only on positive class.

$$Jaccard = \frac{TP}{TP + FP + FN}$$

$$Jaccard = \frac{35}{35 + 10 + 5}$$

$$= \frac{35}{50} = \mathbf{0.700}$$



Benefit:
Higher value means more overlap between actual and predicted 5s (less FP and FN).

Classification Metrics (cont'd)

- Youden's Index $(YI) = TPR + TNR - 1$: It evaluates the discriminative power of the test.
- Matthews Correlation Coefficient $(MCC) = \frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$: It represents the correlation between the observed and predicted classifications.
- F-measure $= DSC = \frac{2TP}{(2TP+FP+FN)}$: It represents the harmonic mean of precision and recall.
- Jaccard $= \frac{TP}{(TP+FP+FN)}$: It is sensitive to changes in data distributions.

Precision and Recall

- Precision is the **accuracy of the positive predictions** of the classifier.
- Recall is the **ratio of positive instances that are correctly detected** by the classifier.
- Precision is typically used along with recall.
- It is often convenient to combine precision and recall into a single metric called the F1 score, in particular if you need a simple way to compare two classifiers.
- The F1 score is the **harmonic mean of precision and recall.**

Precision, Recall, and F1-Score
with a Numerical Example

Example: Digit "5" Classification

Suppose we trained a binary classifier to detect the digit 5 from the MNIST dataset.
We evaluate it on a test set of 200 images.

Some test images



Class: 5 (Positive) | Class: Not 5 (Negative)

Confusion Matrix

		Predicted Class: Predicted 5 (Positive)	Predicted Class: Predicted Not 5 (Negative)
Actual Class	Actual 5 (Positive)	80 (True Positive) TP	10 (False Negative) FN
	Actual Not 5 (Negative)	15 (False Positive) FP	95 (True Negative) TN

Precision

Among all instances predicted as 5, how many are actually 5?

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Precision} = \frac{80}{80 + 15} = 0.842$$

Precision = 0.842 (84.2%)

Recall

Among all actual 5s, how many are correctly detected as 5?

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Recall} = \frac{80}{80 + 10} = 0.889$$

Recall = 0.889 (88.9%)

F1-Score

The harmonic mean of Precision and Recall.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

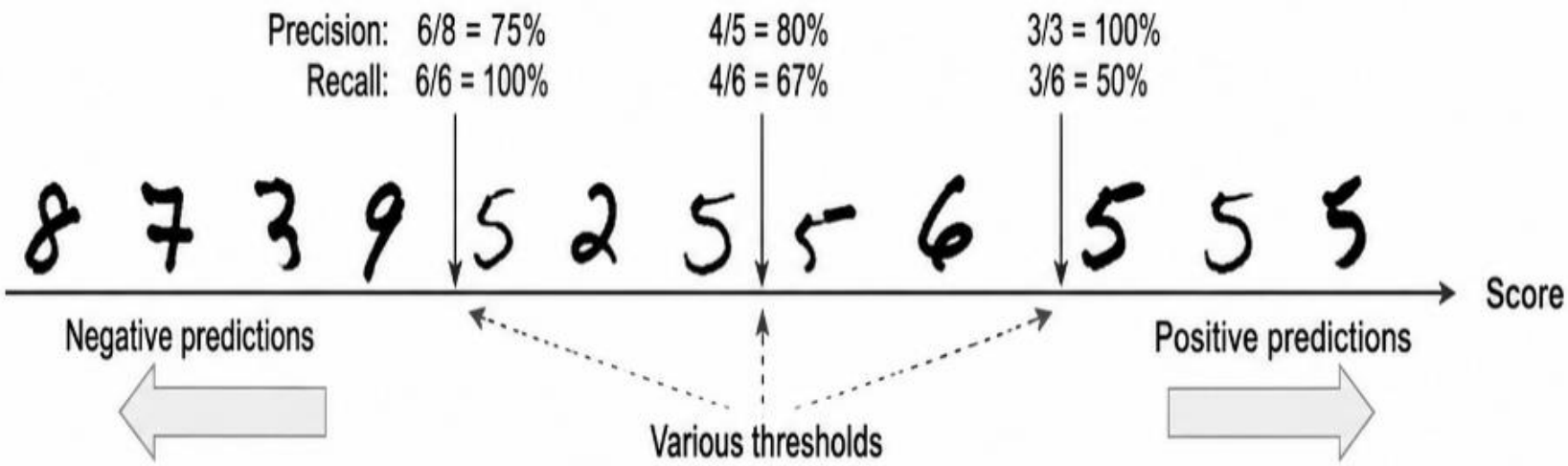
$$F1 = 2 \times \frac{0.842 \times 0.889}{0.842 + 0.889} = 0.865$$

F1-Score = 0.865 (86.5%)

Precision, Recall, and F1 Score Code

```
>>> from sklearn.metrics import precision_score, recall_score
>>> precision_score(y_train_5, y_train_pred) # == 4096 / (4096 + 1522)
0.7290850836596654
>>> recall_score(y_train_5, y_train_pred) # == 4096 / (4096 + 1325)
0.7555801512636044
>>> from sklearn.metrics import f1_score
>>> f1_score(y_train_5, y_train_pred)
0.7420962043663375
```

- Whereas the regular mean **treats all values equally**, the harmonic mean **gives much more weight to low values.**
- As a result, the classifier will only get a high F1 score if both **recall and precision are high.**



Threshold 1 (low)

Predict:	N	N	N	N	P	P	P	P	P	5	P	P
	8	7	3	9	5	2	5	5	6	5	5	5

Confusion Matrix

Actual	Predicted: Negative (not 5)	Predicted: Positive (5)
Negative (not 5)	4 (TN)	2 (FP)
Positive (5)	0 (FN)	6 (TP)

Precision = 6/8 = 75%

Recall = 6/6 = 100%

Threshold 2 (medium)

Predict:	N	N	N	N	P	N	P	P	N	P	P	P
	8	7	3	9	5	2	5	5	6	5	5	5

Confusion Matrix

Actual	Predicted: Negative (not 5)	Predicted: Positive (5)
Negative (not 5)	5 (TN)	1 (FP)
Positive (5)	2 (FN)	4 (TP)

Precision = 4/5 = 80%

Recall = 4/6 = 67%

Threshold 3 (high)

Predict:	N	N	N	N	N	N	N	P	N	P	P	P
	8	7	3	9	5	2	5	5	6	5	5	5

Confusion Matrix

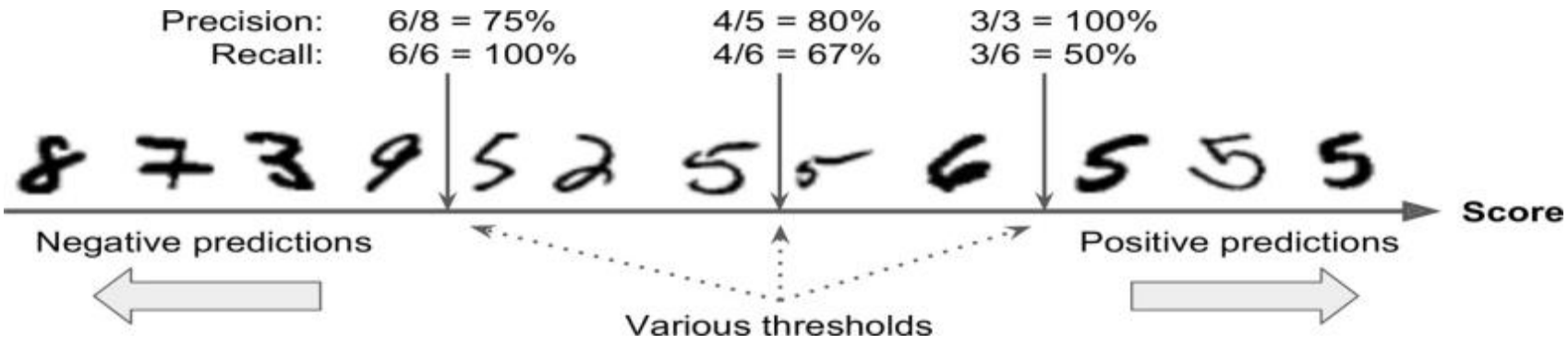
Actual	Predicted: Negative (not 5)	Predicted: Positive (5)
Negative (not 5)	6 (TN)	0 (FP)
Positive (5)	3 (FN)	3 (TP)

Precision = 3/3 = 100%

Recall = 3/6 = 50%

Precision/Recall Trade-off

- Increasing precision reduces recall, and vice versa. This is called the **precision/recall trade-off.**
- SGDClassifier computes a score based on a **decision function.** If that score is **greater than a threshold,** it assigns the instance to the **positive class;** otherwise it assigns it to the **negative class.**



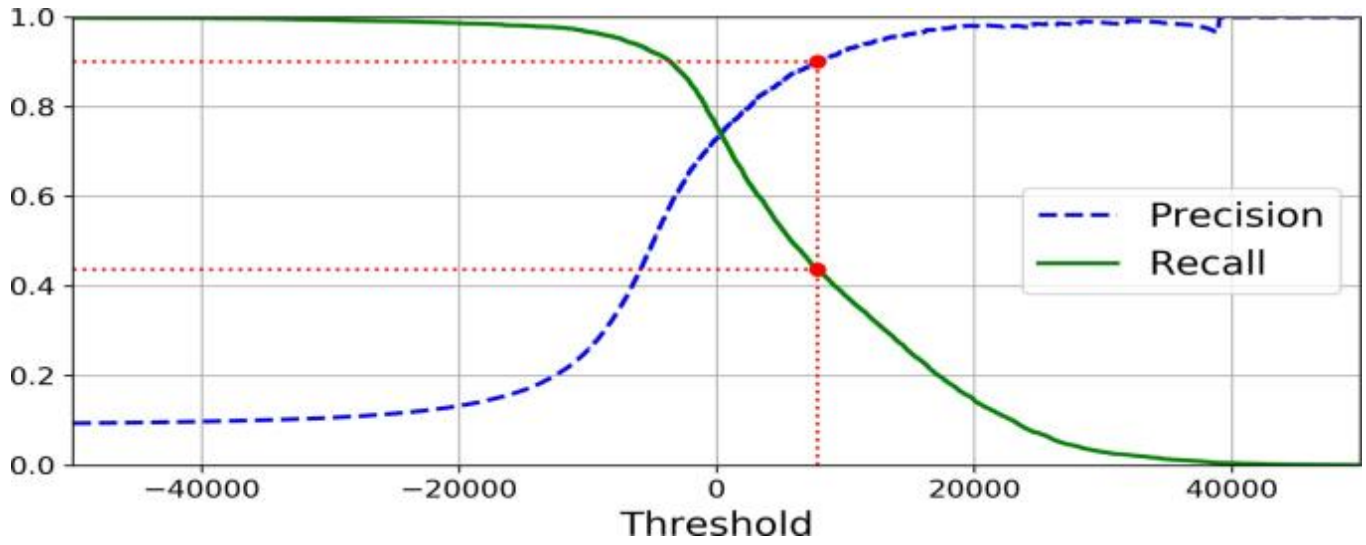
Precision/Recall Trade-off (cont'd)

- Scikit-Learn **does not let you set the threshold directly**, but it does **give you access to the decision scores** that it uses to make predictions.

- decision_function() **returns a score for each instance**, and then use any threshold you want to make predictions based on those scores.

```
>>> y_scores = sgd_clf.decision_function([some_digit])
>>> y_scores
array([2412.53175101])
>>> threshold = 0
>>> y_some_digit_pred = (y_scores > threshold)
array([ True])
```

Precision and Recall versus the Decision Threshold

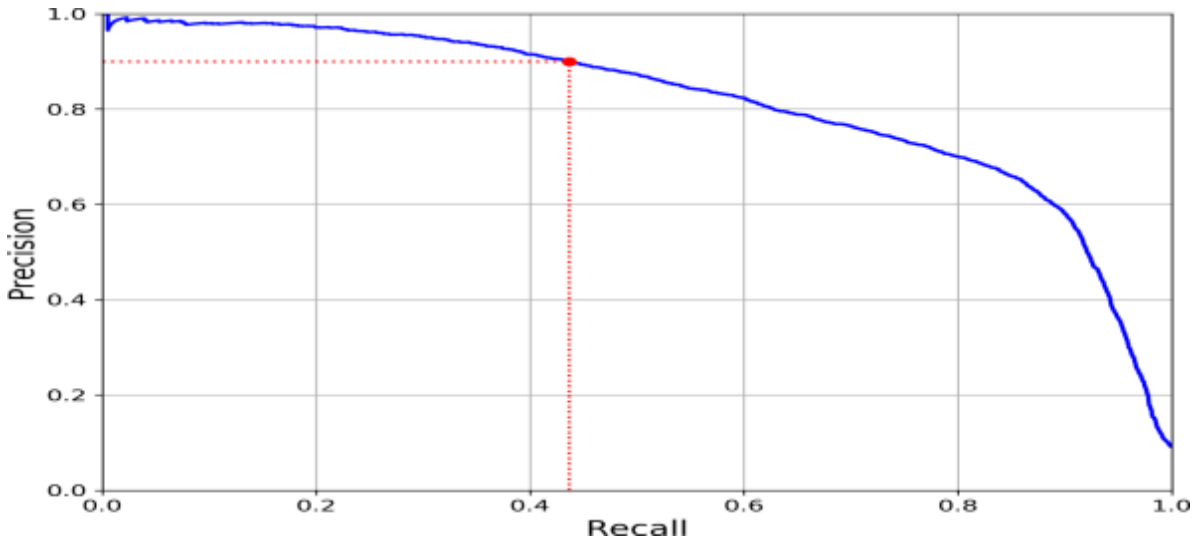


precision_recall_curve() computes precision and recall for **all possible thresholds.**

```
from sklearn.metrics import precision_recall_curve

precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)
```

Precision versus Recall



- Another way to select a good precision/recall trade-off is to plot precision directly against recall.
- You can see that precision really starts to fall sharply around 80% recall.
- The choice depends on your project.

The ROC Curve: Complete Guide

Visualizing the Trade-off Between True Positives and False Positives

The ROC curve shows how well a binary classifier can distinguish between the two classes.

1 What is the ROC Curve?

- The **Receiver Operating Characteristic (ROC)** curve is a common tool used with binary classifiers.
- It plots the **True Positive Rate (TPR)** (also called **Recall** or **Sensitivity**) against the **False Positive Rate (FPR)** for different classification thresholds.
- It shows the trade-off between correctly detecting positives and incorrectly classifying negatives.

2 Key Metrics and Formulas

True Positive Rate (TPR) (Recall / Sensitivity)

$$TPR = \frac{TP}{TP + FN}$$

Proportion of actual positive samples that are correctly classified.

False Positive Rate (FPR)

$$FPR = \frac{FP}{FP + TN}$$

Proportion of actual negative samples that are incorrectly classified as positive.

Specificity (True Negative Rate)

$$Specificity = \frac{TN}{TN + FP} = 1 - FPR$$

Precision (PPV)

$$Precision = \frac{TP}{TP + FP}$$

Negative Predictive Value (NPV)

$$NPV = \frac{TN}{TN + FN}$$

3 Example Confusion Matrix

Binary classification: Detecting digit "5"

		Predicted Class: Predicted 5 (Positive)	Predicted Class: Predicted Not 5 (Negative)
Actual Class	Actual 5 (Positive)	TP = 80	FN = 10
	Actual Not 5 (Negative)	FP = 15	TN = 95

4 Calculate the Metrics (using the example)

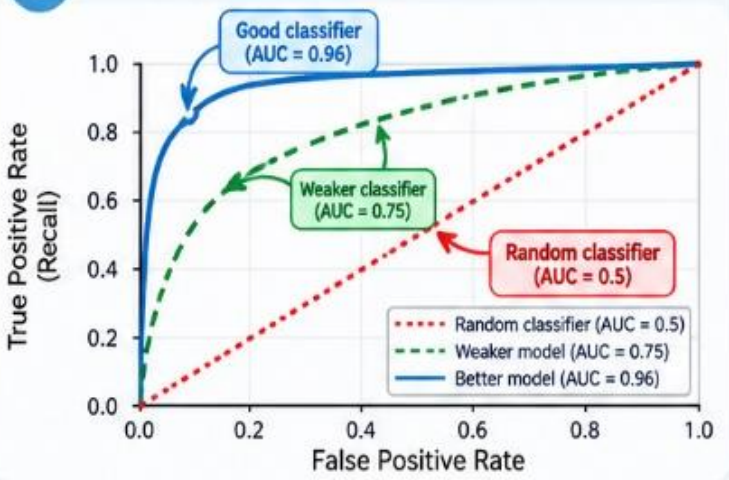
TPR (Recall)
$= 80 / (80 + 10) = 0.889$ **(88.9%)**

FPR
$= 15 / (15 + 95) = 0.136$ **(13.6%)**

Precision (PPV)
$= 80 / (80 + 15) = 0.842$ **(84.2%)**

Specificity (TNR)
$= 95 / (95 + 15) = 0.864$ **(86.4%)**

5 The ROC Curve Plot



6 Interpreting the ROC Curve

Top-Left Corner (0, 1)

Ideal classifier
- TPR = 1 (100% recall)
- FPR = 0 (no false positives)
- AUC = 1

Diagonal Line

Random classifier
- TPR = FPR
- AUC = 0.5

Closer to Top-Left is Better

- The curve should be as far away from the diagonal line as possible.
  A larger AUC indicates a better classifier.

7 Area Under the Curve (AUC)

- AUC is a single number that summarizes the overall performance of the classifier.
- A perfect classifier has AUC = 1.
- A purely random classifier has AUC = 0.5.
- Higher AUC means better ability to distinguish between the two classes.

AUC = 1 Perfect!

Rule of Thumb

You should prefer the precision-recall (PR) curve whenever the positive class is rare or when you care more about the false positives than the false negatives.

The ROC Curve

- The receiver operating characteristic (ROC) curve is common tool used with binary classifiers.
- The ROC curve plots the true positive rate (another name for recall) against the false positive rate (FPR).
- The ROC curve plots **sensitivity (recall)** versus **1 – specificity**.

```
from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

Area Under the Curve (AUC)

- One way to compare classifiers is to measure AUC.
- A perfect classifier will have a ROC AUC **equal to 1**, whereas a purely random classifier will have a ROC AUC **equal to 0.5**.

```
>>> from sklearn.metrics import roc_auc_score
>>> roc_auc_score(y_train_5, y_scores)
0.9611778893101814
```

Rule of Thumb

You should prefer the PR curve whenever the **positive class is rare** or when you **care more about the false positives** than the false negatives.

Area Under the Curve (AUC)

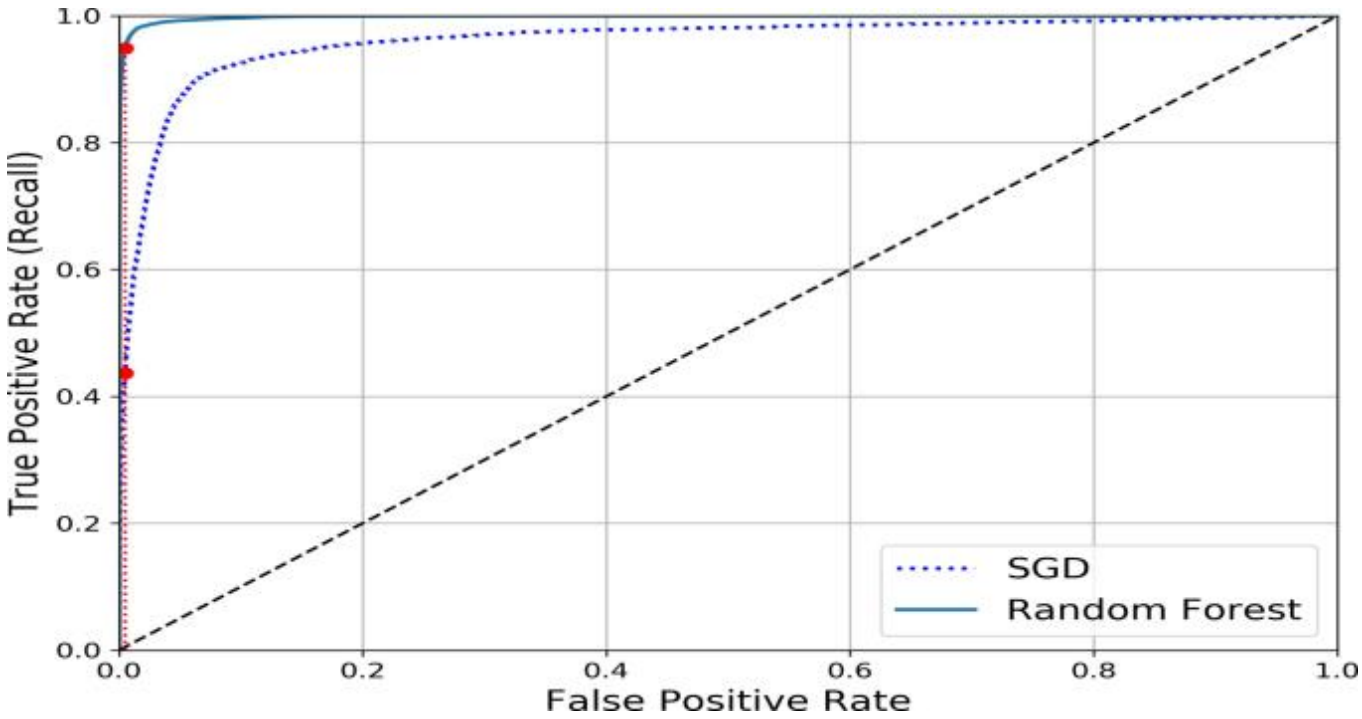
- One way to compare classifiers is to measure AUC.
- A perfect classifier will have a ROC AUC **equal to 1**, whereas a purely random classifier will have a ROC AUC **equal to 0.5**.

```
>>> from sklearn.metrics import roc_auc_score
>>> roc_auc_score(y_train_5, y_scores)
0.9611778893101814
```

Rule of Thumb

You should prefer the PR curve whenever the **positive class is rare** or when you **care more about the false positives** than the false negatives.

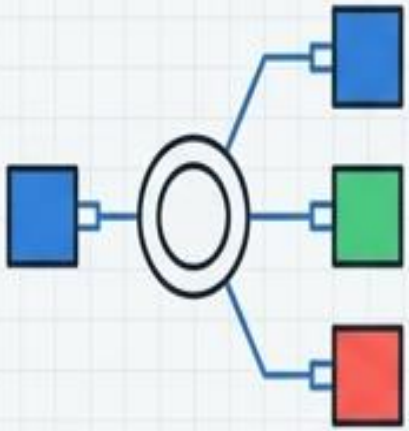
ROC Curve for the Random Forest Classifier



The RandomForestClassifier class does not have a **decision_function()** method. Instead, it has a **predict_proba()** method.

The Complex Reality: Multiclass Classification

The Challenge: Distinguishing between more than two categories (e.g., digits 0 through 9).

Native Multiclass	Strict Binary
Algorithms inherently capable of handling multiple distinct classes out-of-the-box without architectural changes.	Algorithms designed purely for binary Yes/No decisions that require strategic adaptation or wrappers to scale.
	
• Random Forest • Naive Bayes • SGD Classifiers	• Logistic Regression • Support Vector Machines (SVM)

Multiclass (Multinomial) Classification

- It can distinguish between more than two classes.
- Some algorithms, such as SGD classifiers, Random Forest classifiers, and naive Bayes classifiers, are capable of handling multiple classes natively.
- Others, such as Logistic Regression or Support Vector Machine classifiers, are strictly binary classifiers.
- There are various strategies that you can use to perform **multiclass classification** with **multiple binary classifiers.**

Multiclass Classification: OvR vs. OvO

Goal

Classify digit images into **10 classes** (from 0 to 9).



Both strategies use **binary classifiers**, but they differ in how they train and combine them for multiclass prediction.

1 One-versus-the-rest (OvR)

Train **one binary classifier for each class:** one to detect a specific digit vs. all other digits.


Class 0 → Binary Classifier 0 → **Digit 0** vs. Not 0 (1,2,3,4,5,6,7,8,9)


Class 1 → Binary Classifier 1 → **Digit 1** vs. Not 1 (0,2,3,4,5,6,7,8,9)


Class 9 → Binary Classifier 9 → **Digit 9** vs. Not 9 (0,1,2,3,4,5,6,7,8)

Prediction (OvR)


Test image

Classifier 0	-2.1
Classifier 1	0.5
...	
Classifier 5	**3.6**
...	...
Classifier 9	-1.0

Get the decision score from each classifier and select the class with the **highest score.**

Predicted class: 5

2 One-versus-one (OvO)

Train a binary classifier for **every pair of digits.**


0 vs. 1 → Binary Classifier (0 vs 1)
0 vs. 2 → Binary Classifier (0 vs 2)
1 vs. 2 → Binary Classifier (1 vs 2)
...
8 vs. 9 → Binary Classifier (8 vs 9)

If there are **N** classes, you need to train $N \times (N-1) / 2$ binary classifiers.

For MNIST ($N = 10$): $10 \times 9 / 2 = 45$ binary classifiers.

Prediction (OvO)


Test image

Classifier (0 vs 1)	0
Classifier (0 vs 2)	2
...	
Classifier (5 vs 7)	**5**
...	
Classifier (8 vs 9)	9

Each classifier votes for one of the two classes. The final class is the one with the **most votes.**

Predicted class: 5 (majority vote)

3 Key Points and Comparison

Aspect	One-vs-Rest (OvR)	One-vs-One (OvO)
Number of classifiers	N (10 for MNIST)	$N \times (N-1) / 2$ (45 for MNIST)
Training data per classifier	All training data (positive vs rest)	Only data from the two selected classes
Prediction	Highest decision score	Majority vote
Training time	Slower (each classifier uses all data)	Faster (each classifier uses a small subset)
Usually preferred	Most binary classifiers (like Logistic Regression, SVM)	Some algorithms (e.g., SVM) that scale poorly with data size

4 When to Use Each Strategy?

- **OvR is preferred** for most binary classification algorithms.
- **OvO is preferred** for algorithms that scale poorly with the size of the training set (e.g., SVM), because each classifier is trained on a smaller subset of the data.
- In practice, many implementations choose automatically.

5 Scikit-Learn Implementation

- Scikit-Learn detects when you use a binary classification algorithm for a multi-class task and **automatically runs OvR or OvO,** depending on the algorithm.
- To force a specific strategy, use:

```
from sklearn.multiclass import OneVsRestClassifier,
    OneVsOneClassifier
```

The One-versus-the-rest (OvR) (One-versus-all) Strategy

- One way to create a system that can classify the digit images into 10 classes (from 0 to 9) is to **train 10 binary classifiers,** one for each digit.

- You get the decision score from each classifier for that image and you select the class whose classifier outputs the highest score.

The One-versus-one (OvO) Strategy

- Train a binary classifier for **every pair of digits**: one to distinguish 0s and 1s, another to distinguish 0s and 2s, another for 1s and 2s, and so on.
- If there are N classes, you need to train $N \times (N-1)/2$ classifiers $\Longrightarrow$ **45 binary classifiers for MNIST dataset.**
- The main advantage of OvO is that each classifier **only needs to be trained on the part of the training set** for the two classes that it must distinguish.

Notes 1

- Some algorithms, such as Support Vector Machine classifiers, **scale poorly with the size of the training set.**
- For these algorithms, OvO is preferred because it is faster to train many classifiers on small training sets than to train few classifiers on large training sets.
- For most binary classification algorithms, however, **OvR is preferred.**

Notes 2

- Scikit-Learn detects when you try to use a binary classification algorithm for a multi-class classification task, and it automatically runs OvR or OvO, **depending on the algorithm.**

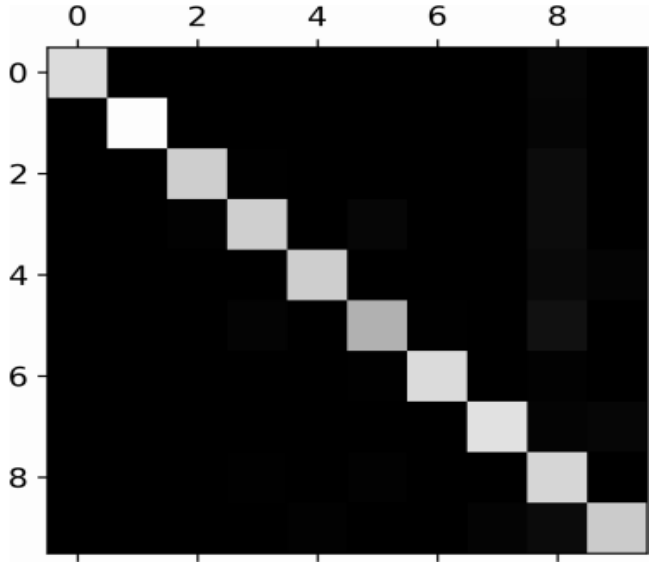
- If you want to force Scikit-Learn to use **one-versus-one** or **one-versus-the-rest**, you can use the **OneVsOneClassifier** or **OneVsRestClassifier** classes.

Error Analysis

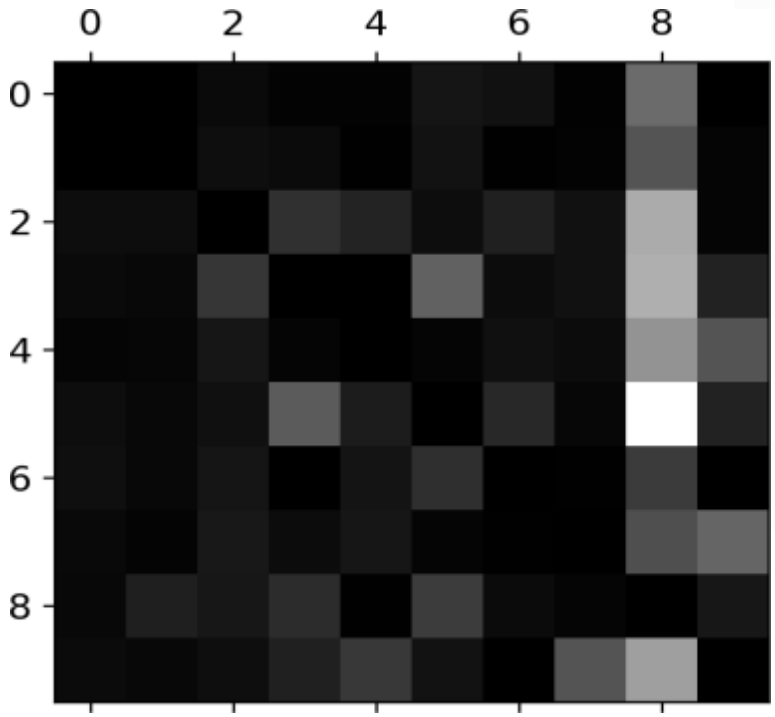
- Assume that you **have found a promising model** and you want to find ways to **improve it.** One way to do this is to analyze the types of errors it makes.
- Look at the confusion matrix. It's often more convenient to look at an image representation of the confusion matrix.
- Analyzing individual errors can also be a good way to gain insights on what your classifier is doing and why it is failing, but it is more difficult and time-consuming.

```
>>> y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
>>> conf_mx = confusion_matrix(y_train, y_train_pred)
>>> conf_mx
array([[5578,    0,   22,    7,    8,   45,   35,    5,  222,    1],
       [   0, 6410,   35,   26,    4,   44,    4,    8,  198,   13],
       [  28,   27, 5232,  100,   74,   27,   68,   37,  354,   11],
       [  23,   18,  115, 5254,    2,  209,   26,   38,  373,   73],
       [  11,   14,   45,   12, 5219,   11,   33,   26,  299,  172],
       [  26,   16,   31,  173,   54, 4484,   76,   14,  482,   65],
       [  31,   17,   45,    2,   42,   98, 5556,    3,  123,    1],
       [  20,   10,   53,   27,   50,   13,    3, 5696,  173,  220],
       [  17,   64,   47,   91,    3,  125,   24,   11, 5421,   48],
       [  24,   18,   29,   67,  116,   39,    1,  174,  329, 5152]])
```

```
plt.matshow(conf_mx, cmap=plt.cm.gray)
plt.show()
```



This confusion matrix **looks pretty good**, since most images are on the main diagonal, which means that they were classified correctly.



Actual Class	Total Images	Misclassified Images	Error Rate
Digit 3	1,000	100	10%
Digit 5	200	60	30%
Digit 8	500	100	20%

You need to **divide each value** in the confusion matrix by the number of images in the corresponding class so that you can compare error rates instead of absolute numbers of errors.

Confusion Matrix Analysis

1. You could try to gather more training data for digits that look like 8s (but are not) so that the classifier can learn to distinguish them from real 8s.

2. You could engineer new features that would help the classifier, for example, writing an algorithm to count the number of closed loops.

3. You could preprocess the images.

Multilabel Classification

One instance can belong to multiple classes at the same time

Main Idea

- In some cases, you may want your classifier to output multiple classes for each instance.
- For example, what should it do if it recognizes several people in the same picture?

Example: Image with Multiple People



Output (Multilabel)

Person	☑
Car	☐
Dog	☐
Cat	☐
Hat	☑
Backpack	☑

Multioutput Classification

Each label can be multiclass (more than two possible values)

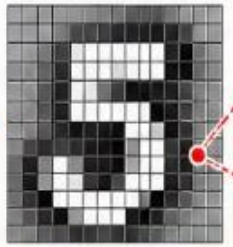
Main Idea

- It is called multioutput–multiclass classification (or simply multioutput classification).
- It is a generalization of multilabel classification where each label can be multiclass (i.e., it can have more than two possible values).
- For MNIST dataset, the classifier's output is multilabel (one label per pixel) and each label can have multiple values (pixel intensity ranges from 0 to 255).

Example: MNIST Image (Pixel-wise Output)



28 × 28 image (784 pixels)



Each pixel is a separate **label**

Output is multilabel (one label per pixel)

Each label can have multiple values (pixel intensity from 0 to 255).

Pixel intensity (values)

255 (dark/black)

...

0 (white)

Evaluation for Multilabel Classification

Measure the **F1 score** for each individual label, then compute the average.

Example

Suppose we have 3 labels: Person, Hat, Backpack

Label	Precision	Recall	F1 Score
Person	0.90	0.80	0.85
Hat	0.75	0.60	0.67
Backpack	0.80	0.70	0.75

Average F1 Score (Macro Average)

$$F1_{avg} = \frac{F1_{person} + F1_{hat} + F1_{backpack}}{3}$$

$$= \frac{0.85 + 0.67 + 0.75}{3} = \mathbf{0.76}$$

Key Takeaways

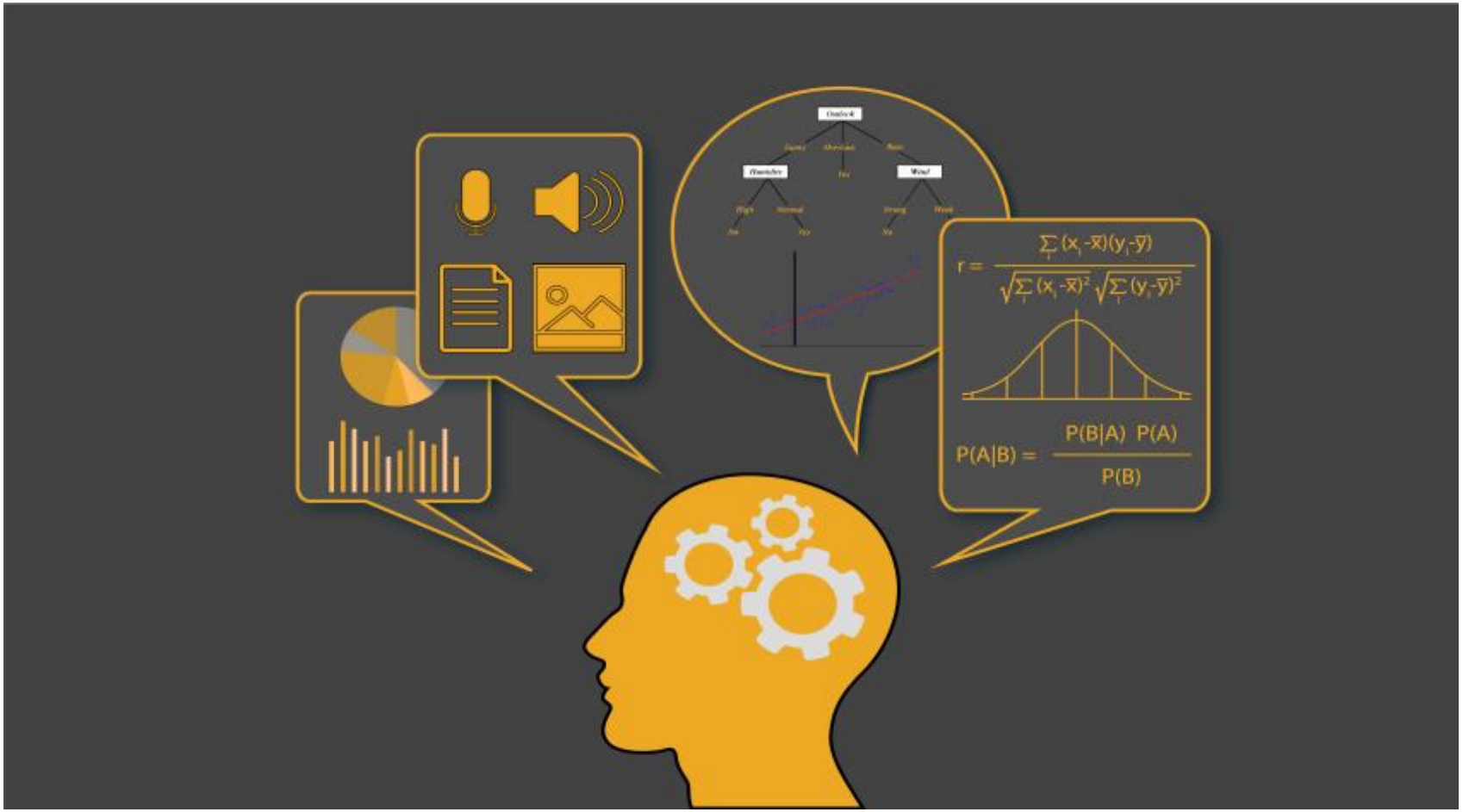
- **Multilabel**: multiple classes per instance (binary labels).
- **Multioutput**: each label can be multiclass (more than two possible values).
- For multilabel classification, compute the F1 score for each label and take the average.

Multilabel Classification

- In some cases, you may want your classifier to output multiple classes for each instance.
- For example, what should it do if it recognizes several people in the same picture?
- To **evaluate a multilabel classifier**, measure the F1 score for each individual label, then simply compute the average score.

Multioutput Classification

- It is called multioutput–multiclass classification (or simply multioutput classification).
- t is simply a generalization of multilabel classification where each label can be multiclass (i.e., it can have more than two possible values).
- For MNIST dataset, the classifier's output is multilabel (one label per pixel) and each label can have multiple values (pixel intensity ranges from 0 to 255).


$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}$
$P(A|B) = \frac{P(B|A)\ P(A)}{P(B)}$